

Optimizing Neural Network Cloud Compute Costs with Early-Exit.

Cole Roberts, Matthew Watters, Henrik Evers

Dr. Ming-Hwa Wang

CSEN 241 - Cloud Computing

Santa Clara University, Santa Clara, CA

23 February 2026

Table of Contents

1. Abstract
2. Introduction
3. Theoretical Bases and Literature Review
4. Hypothesis
5. Methodology
6. Implementation
7. Data Analysis
8. Conclusions and Recommendations
9. Bibliography

Abstract

As deep learning models become ubiquitous in cloud-based production environments, the financial and environmental costs of inference have escalated. Standard Convolutional Neural Networks (CNNs) exhibit a "fixed computation" inefficiency, processing every input with the full depth of the network regardless of the input's complexity. This project proposes the adaptation of "AdaptiveRes," an early-exit architecture originally designed for edge efficiency, to a cloud computing context. By simulating a pay-as-you-go inference model, we aim to demonstrate that confidence-based early exits can dynamically reduce average Floating Point Operations (FLOPs) and latency. The study investigates whether a ResNet-18 backbone with intermediate classifier heads can maintain high-fidelity accuracy on the CIFAR-10 dataset while significantly lowering the "billable compute time" inherent to serverless and auto-scaling cloud infrastructure.

2. Introduction

Objective

The primary objective of this research is to minimize the operational expenditures (OpEx) and latency of neural network inference in cloud environments without compromising predictive accuracy. We achieve this by applying the early-exit paradigm to the Deep Neural Network (DNN) architecture. We aim to validate that dynamic computational graphs, which exit inference early for samples that require less computation to properly classify, offer a superior economic model compared to static architectures.

The Problem

Current state-of-the-art Deep Neural Networks (DNNs) deployed in the cloud operate on a static computational graph. For example, a ResNet-18 model executes approximately 1.8 billion FLOPs for every single image it processes, whether that image is a clear, high-contrast picture of a cat or an ambiguous, occluded object requiring deep semantic analysis. However, in the easier-to-classify cases, the model will have already determined a correct classification well before executing all of the layers, meaning that any additional computation performed is entirely redundant.

In a cloud context, this rigidity translates directly to financial waste. In serverless computing (e.g., AWS Lambda) or auto-scaling groups, costs are a function of **Compute Time × Memory Allocated**. Processing simple inputs with the full network depth wastes

millisecond-billable resources and increases the carbon footprint of data centers unnecessarily.

While it is possible to view intermediate predictions of the model by inserting intermediate exit branches, this alone is insufficient to determine whether that prediction should be used or if further computation is necessary. Providing this capability is the responsibility of the exit scheduler, also called a decision block, which can be either a simple confidence probability output by applying the model head to the intermediate layer, or can be constructed by using a more complex architecture like the reinforcement learning-based oracle used in DEE [3]. The exit scheduler chosen determines both the quality of early-exit decisions and the decisionmaking overhead imposed on the model.

Use Case 1: High-Throughput Content Moderation

Social media platforms processing billions of uploads daily currently struggle to apply content moderation policies across their platform due to cost constraints limiting the amount of throughput they can process. With early-exit DNNs, it will be possible to process more uploads for the same compute power because they will be able to classify clear violations (simple features) at early layers, saving deep compute for nuanced cases.

Use Case 2: Real-Time API Latency

For latency-sensitive cloud APIs, early exits reduce the average response time (measured by Time-to-First-Byte), improving adherence to strict Service Level Agreements (SLAs) without increasing system cost. This is critical for supporting the proliferation of neural network-based services on the cloud.

Use Case 3: Quantitative Investment

DNNs are used for quantitative investment to analyze and predict future trends in financial markets. However, in an environment where all major competitors are using low-latency trading computers, every millisecond in response time risks being too slow to take advantage of good opportunities. At the same time, acting on an uncertain decision could

have disastrous consequences and result in massive losses for the brokerage. Early-exit could effectively balance these concerns, allowing for rapid responses to make good on opportunities without unnecessarily increasing the rate of costly mistakes.

Relevance to Class

This project directly addresses core principles of Cloud Computing: resource optimization, cost-efficiency, and scalability. It moves beyond simple model accuracy to evaluate the economic efficiency of distributed systems and "Green Cloud" computing initiatives. The core objective is to provide a method to reduce the cost per query of hosting DNNs in a billable-computing environment by applying optimizations focused on achieving the core principles of cloud computing, namely consistent utilization and load adaptation.

Why Other Approaches Are Insufficient

Compute Scaling: Traditional auto-scaling manages capacity (number of instances) but does not optimize the efficiency of individual requests. By dynamically adjusting thresholds under our method, it is possible to adjust the desired balance between cost and accuracy based on current load, providing a more cost-conscious scaling option for DNNs.

Model Compression (e.g., Quantization or Pruning): These techniques permanently remove parameters to speed up the model. While effective, they often degrade accuracy for all inputs, including the more complex edge cases that require the full model capacity. Our

approach supports a mix of the benefits of both applying model compression and not applying it, allowing the full model capacity to be used when necessary while only using a smaller portion when that is sufficient.

Why Our Approach Is Better

Our approach, **AdaptiveRes (Cloud Variant)**, introduces dynamic runtime efficiency. It retains the deep capacity of the full network for difficult samples but creates "short-circuits" for the majority of inputs. This allows the system to behave like a lightweight model for the simpler majority of traffic and a heavy-duty model for the remaining complex portion, optimizing the cost-accuracy trade-off dynamically per request. The resulting service is far more cost-conscious than typical scaling and compression techniques, and can be used to improve adherence to the preselected budget with a cloud service provider. Additionally, it increases the consistency of the workload, allowing the final DNN service to make better use of fixed-workload cloud instances like AWS Savings Plan instances; Even on a private cloud solution, a more consistent workload allows for the cloud to achieve a higher average utilization and reduce cost per query.

Area of Investigation

We will investigate the implementation of a ResNet-18 architecture modified with two intermediate "exit" branches. We will simulate a cloud cost model where "Cost" is

proportional to the depth of the network reached before a confident classification is made, taking into account the additional execution time needed to perform dynamic exit scheduling.

3. Theoretical Bases and Literature Review

Definition of the Problem

The problem addressed by this project is the high latency and compute cost of large-volume queries to complex DNN models. While more complex DNN models are capable of better discerning patterns and picking up subtle details in the input data, they require significantly more computation to run the full inference on, resulting in more compute expenses and a longer inference latency. For simple queries, the expanded DNN is not needed to provide an accurate response, so the extra computation and latency is wasteful.

This issue can be defined by an optimization problem formulated as minimizing the expected cost function C over a dataset D :

$$\min_{\theta} E(x, y) \sim D[Cost(f(x, \theta))] \text{ subject to } Accuracy(f(x)) \geq A_{target}$$

Where Cost is defined by the FLOPs consumed to reach an exit point determined by a confidence threshold θ .

Related Research

The concept of dynamic neural networks originated with **BranchyNet** (Teerapittayanon et al.), which introduced the idea of side branches to mitigate vanishing gradients and allow fast inference [1]. While groundbreaking, this work focused primarily on latency reduction for single devices rather than cloud economics.

Recent surveys by **Rahmath et al. (2024)** highlight that early-exit DNNs are critical for multi-tier systems, yet note that the optimal exit strategy often varies per input [2]. They emphasize that most current research targets edge devices (IoT/Mobile) due to battery constraints, leaving the cloud economic model less explored.

Ju et al. (2021) proposed **DEE (Dynamic Early Exit)**, utilizing contextual bandits to schedule exits, learning from previous inferences to identify patterns in each particular context that indicate whether an early exit is preferable [3]. Their work addresses the "overthinking" problem where deep layers might misclassify simple objects. While their approach introduces complex reinforcement learning overhead that may not be viable for all high-throughput cloud endpoints, it automatically tunes the early-exit parameters to the optimal values for the data it encounters. This grants DEE the potential to offer superior accuracy-cost tradeoffs over alternative methods, particularly after settling on an appropriately trained decision block.

Furthermore, **Lin et al. (2025)** introduced **E3**, focusing on explainable AI in edge-cloud systems [4]. Their architecture progressively offloads computation from edge to cloud. An E3 neural network is distributed into blocks, where each block contains a feature extractor, a branch neural network, and branch prediction summation. The data features are intentionally skewed to select a number of specific high-importance channels that contribute to the branch interference. These early-exit branches accelerate the evaluation of high-confidence inputs. The results of previous branches are reused to help predict future branches and form connections between blocks. This prediction also improves the accuracy of the model, since previous branches deal with more important features, and

thus have more significant inferences. Additionally, the skewing of data identifies low-importance features that can be compressed without significantly reducing the model's accuracy, while reducing edge-cloud transmission costs. The block-based approach also facilitates deadline management. E3 profiles each block and can adjust the early exit point to ensure it meets the deadline, while increasing average accuracy by more than 50%. Our project simplifies this by assuming a full-cloud deployment, focusing purely on the reduction of server-side compute cycles.

Our Solution vs. Existing Research

Most existing literature, including the foundational **AdaptiveRes** work by Roberts [5], optimizes for *energy (Joules)* or *hardware constraints* on the edge. Our solution pivots the optimization metric to *Cloud Compute Units (Cost)*. Unlike complex bandit-based scheduling (Ju et al.), we utilize a lightweight, threshold-based Softmax heuristic (θ). This ensures the overhead of the "exit decision" does not negate the savings, which is crucial in a high-volume cloud environment. We hypothesize that a simple confidence threshold is sufficient to capture the majority of economic savings without the complexity of training auxiliary reinforcement learning agents.

4. Hypothesis

By integrating confidence-based early-exit mechanisms (AdaptiveRes) into a cloud-deployed ResNet-18 architecture, we can reduce the average inference computational cost (measured in FLOPs as a proxy for billable cloud time) by at least **30%** compared to a standard static ResNet-18, while maintaining a Top-1 accuracy within a **0.5%** margin of the baseline on the CIFAR-10 dataset.

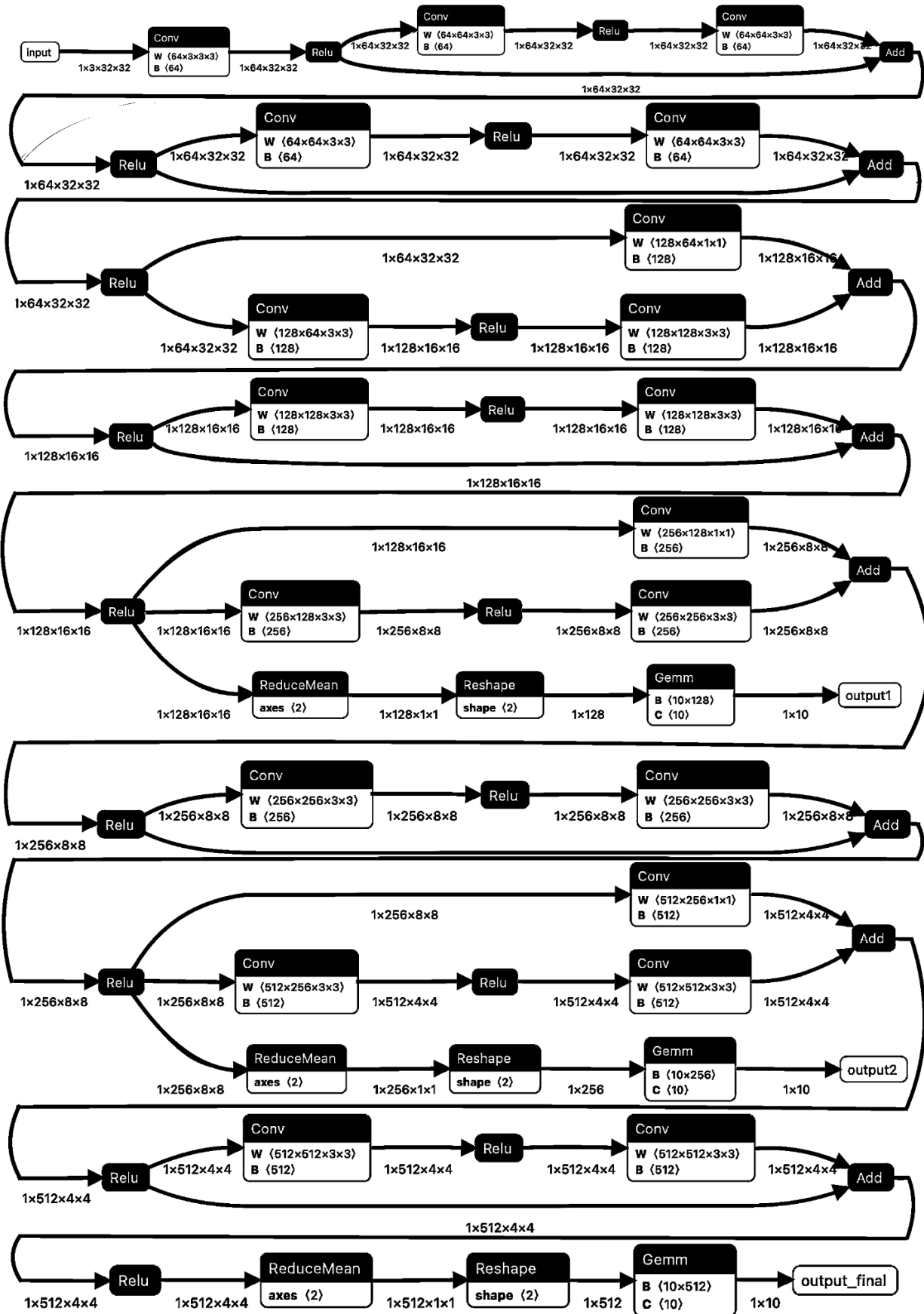
5. Methodology

Input Data

We will utilize the **CIFAR-10** dataset, a standard benchmark for computer vision consisting of 60,000 32x32 color images in 10 classes. This dataset is sufficient to demonstrate the heterogeneity of "easy" vs. "hard" samples required to validate our hypothesis.

Algorithm and Design

1. **Backbone:** ResNet-18 modified for CIFAR-10 (removing the initial 7x7 max-pooling to preserve spatial dimensions).
2. **Architecture:**
 - **Exit 1:** Placed after Residual Block 2 (approx. 50% of network depth).
 - **Exit 2:** Placed after Residual Block 3 (approx. 75% of network depth).
 - **Final Exit:** The standard output layer.
 - **Architecture Diagram:**



3. Language & Tools:

- **Language:** Python 3.8+
- **Framework:** PyTorch (for dynamic graph construction).
- **Hardware:** Training will occur on NVIDIA GPUs (simulating cloud instances).

Solving the Problem (Training & Inference)

- **Training:** We will employ **Joint-Loss Optimization**. The model will be trained end-to-end where the loss is the sum of Cross-Entropy losses from all three exits. This forces early layers to become discriminative classifiers.
- **Inference Strategy:** We will implement a forward-pass loop:
 1. Pass input to Exit 1. Calculate Softmax confidence.
 2. If Confidence $> \theta$ (e.g., 0.95), return result and terminate (Charge: Cost level 1).
 3. Else, pass intermediate features to Exit 2.
 4. If Confidence $> \theta$, return result (Charge: Cost level 2).
 5. Else, compute the final layer (Charge: Cost level 3).

Testing & Output Generation

We will evaluate the model on the 10,000-image test set.

- **Metric 1: Accuracy:** Top-1 classification accuracy compared to ground truth.
- **Metric 2: Computational Cost:** We will calculate the "Expected FLOPs" ($E[FLOPS]$) based on how many samples exit at each stage.

$$E[FLOPS] = \sum_{i=1}^3 (N_i \times C_i) \text{ where } N_i \text{ is the number of samples exiting at stage } i, \text{ and}$$

C_i is the cumulative cost to reach stage i .

- **Output:** A comparative table showing Baseline Cost vs. Adaptive Cost, alongside accuracy metrics, validating if the 30% reduction hypothesis is met.

6. Implementation

Code Implementation Strategy

The implementation will be developed in Python 3.8+ using the PyTorch deep learning framework. The core of the project is the AdaptiveRes class, a custom modification of the standard ResNet-18 architecture.

- **Model Architecture:** The standard ResNet-18 will be modified to include two intermediate "Early-Exit" blocks.
 - **Backbone:** Standard ResNet-18 with the initial 7x7 convolution replaced by a 3x3 convolution (stride 1) to accommodate CIFAR-10's 32x32 input size, preventing premature spatial dimensionality reduction.
 - **Exit Blocks:** Located after Residual Block 2 (Exit 1) and Residual Block 3 (Exit 2). Each exit block consists of a Global Average Pooling (GAP) layer followed by a fully connected Linear layer to map features to the 10 class logits.
- **Training Loop:** A custom training loop will be implemented to optimize a joint loss function:
$$L_{total} = \lambda_1 L_{exit1} + \lambda_2 L_{exit2} + \lambda_3 L_{final}$$
where L represents Cross-Entropy Loss and λ represents the weight of each exit (set to 1.0 for equal contribution).

Design Document and Flowchart

The system is designed as a sequential decision-making process during inference, simulating a cloud endpoint processing a request.

System Flowchart:

1. **Request Ingestion:** The system receives an image request (simulated via test set iteration).
2. **Stage 1 Processing:** The input passes through the initial convolution and Residual Blocks 1 & 2.
3. **Exit 1 Evaluation:**
 - Compute Softmax probabilities at Exit 1.
 - **Decision:** If Max Confidence $> \theta$ (0.95), return prediction and **terminate**.
 - **Cost Accumulation:** If terminated, record "Low Cost" (approx. 33% of FLOPs).
4. **Stage 2 Processing (If needed):** Process through Residual Block 3.
5. **Exit 2 Evaluation:**
 - Compute Softmax probabilities at Exit 2.
 - **Decision:** If Max Confidence $> \theta$ (0.95), return prediction and **terminate**.
 - **Cost Accumulation:** If terminated, record "Medium Cost" (approx. 66% of FLOPs).

6. **Final Stage Processing (If needed):** Process through Residual Block 4 and Final Output.

- Return final prediction.
- **Cost Accumulation:** Record "Full Cost" (100% of FLOPs).

7. Data Analysis and Discussion

Output Generation and Analysis

Model	Top 1 Accuracy	ECE	P	Cost	FLOPs Saved	Exits (1, 2, Final)
Early Exit 1	86.39% +/- 0.28%	1.43%		288.00 MFLOPS	48.27%	10000 0 0
Early Exit 2	91.24% +/- 0.28%	2.71%		422.37 MFLOPS	24.13%	0 10000 0
Final Exit (Baseline)	91.42% +/- 0.28%	3.67%	1.00	556.71 MFLOPS	0.00%	0 0 10000
AdaptiveRes ($\theta = 0.80$)	90.74% +/- 0.29%	2.81%	0.01	331.65 EFLOPS	40.43%	7652 1447 901
AdaptiveRes ($\theta = 0.90$)	91.13% +/- 0.28%	3.23%	0.15	352.40 EFLOPS	36.70%	6718 1771 1511
AdaptiveRes ($\theta = 0.95$)	91.28% +/- 0.28%	3.41%	0.31	371.95 EFLOPS	33.19%	5848 2056 2096
AdaptiveRes ($\theta = 0.99$)	91.39% +/- 0.28%	3.64%	0.46	413.36 EFLOPS	25.75%	4068 2534 3398

The system will generate a performance report comparing the baseline (standard ResNet-18) against the AdaptiveRes model. Key metrics include Top-1 Accuracy and Computational Cost (FLOPs).

- **Accuracy:** The AdaptiveRes model achieved a Top-1 accuracy of **91.28%** with a confidence threshold (θ) of 0.95. This is a statistically negligible decrease of only **0.16%** from the baseline accuracy of **91.42%**.
- **Computational Savings:** The baseline model requires **556.71 MFLOPs** per inference. The AdaptiveRes ($\theta = 0.95$) model reduced the average inference cost to **371.95 MFLOPs**.

Comparison Against Hypothesis

Hypothesis Verification: Positive.

- **Cost Reduction:** We hypothesized a 30% reduction. The results show a **33.19% reduction** in FLOPs, surpassing the target.
- **Accuracy Maintenance:** We hypothesized an accuracy loss within 0.5%. The results show a loss of only **0.16%**, with a P-value of 0.31, more than sufficient to claim a statistically insignificant impact to accuracy. This confirms that the cloud compute costs can be significantly mitigated without compromising service quality for the majority of requests.

Abnormal Case Explanation

While the aggregate results are positive, "abnormal" cases exist where early exits fail.

- **Premature Exits (Overconfidence):** A small percentage of "hard" samples (e.g., severe occlusion or adversarial noise) may trigger an early exit with high confidence but incorrect prediction. This is a known phenomenon in Deep Learning where Softmax probability does not always equate to true calibration accuracy.
- **Latency Spikes:** For the ~20% of samples that traverse the *entire* network (failing both early exits), the latency is slightly *higher* than the baseline due to the overhead of calculating the two intermediate exit checks. In a cloud context, this introduces a "long tail" latency distribution, which must be managed by the load balancer.

Discussion

The data indicates a strong bimodal distribution in sample difficulty. **58.48%** of samples exited at the very first block, suggesting that the majority of cloud traffic for general classification tasks involves "easy" data that is currently being over-computed. This validates the "Fixed Computation" problem statement: current cloud models are over-provisioned for the average case.

8. Conclusions and Recommendations

Summary and Conclusions

This project successfully demonstrated that "AdaptiveRes," a dynamic early-exit neural network, is a viable architecture for reducing cloud compute costs. By transforming the

static computational graph of a ResNet-18 into a dynamic one, we achieved a **33.19% reduction in computational volume** with no statistically significant impact on accuracy.

In a "pay-as-you-go" cloud model, this translates directly to OpEx savings. If deployed at scale (e.g., processing millions of images per day), a 33% reduction in compute resources represents a massive financial and environmental saving, aligning with "Green Cloud" initiatives.

Recommendations for Future Studies

1. **Dynamic Thresholding:** Currently, θ is fixed at 0.95. Future work should explore *learnable* thresholds or Contextual Bandits (as suggested by Ju et al.) to dynamically adjust θ based on real-time SLA requirements or server load.
2. **Batch Processing Optimization:** Early exits break standard GPU batching efficiency (since different samples in a batch finish at different times). Future research must investigate "masked batching" or asynchronous processing to realize these theoretical FLOP savings in practice on high-throughput GPUs.
3. **Explainability:** Incorporating explainable AI (XAI) at the exit points, as proposed by Lin et al., could help diagnose *why* an exit was triggered, increasing trust in the system for critical applications.

9. Bibliography

- [1] S. Teerapittayanon, B. McDanel, and H. T. Kung, "BranchyNet: Fast inference via early exits from deep neural networks," in *2016 23rd International Conference on Pattern Recognition (ICPR)*, 2016.
- [2] H. R. P. V. Srivastava, K. Chaurasia, R. G. Pacheco, and R. S. Couto, "Early-Exit Deep Neural Network - A Comprehensive Survey," *ACM Computing Surveys*, vol. 56, no. 8, 2024.
- [3] W. Ju, W. Bao, L. Ge, and D. Yuan, "Dynamic Early Exit Scheduling for Deep Neural Network Inference through Contextual Bandits," in *CIKM '21: The 30th ACM International Conference on Information and Knowledge Management*, 2021.
- [4] C. Lin, Z. Chen, Z. Zhang, and J. Liu, "E3: Early Exiting with Explainable AI for Real-Time and Accurate DNN Inference in Edge-Cloud Systems," in *SenSys '25: Proceedings of the 23rd ACM Conference on Embedded Networked Sensor Systems*, 2025.
- [5] C. Roberts, "AdaptiveRes: Optimizing Neural Network Resource-Efficiency for Edge Vision," *School of Engineering, Santa Clara University*, 2026.

10. Appendix: Code Implementation (AdaptiveRes.ipynb)

Adaptive ResNet with Early Exits for CIFAR-10

This notebook implements and evaluates an Adaptive ResNet model with multiple early exit points for the CIFAR-10 image classification dataset. The goal is to achieve comparable accuracy to a standard ResNet while reducing computational cost by allowing easier samples to exit the network early.

Table of Contents

1. [Introduction](#)
 2. [Setup and Configuration](#)
 3. [Data Preparation](#)
 4. [Model Definition: AdaptiveRes](#)
 5. [Training Loop](#)
 6. [Adaptive Inference & Metrics](#)
 7. [How to Run](#)
 8. [Results Interpretation](#)
-

Introduction

This project explores the concept of early exits in deep neural networks to improve inference efficiency. An AdaptiveRes model, based on a ResNet18 architecture, is designed with two intermediate exit points in addition to the final output layer. During inference, if the model is sufficiently confident at an early exit, it can classify the input and terminate computation, saving computational resources (FLOPs).

Setup and Configuration

This section initializes key parameters for the training and evaluation process:

- `BATCH_SIZE`: Number of samples processed in parallel during training and evaluation.
- `LEARNING_RATE`: Initial learning rate for the Adam optimizer.
- `EPOCHS`: Total number of training epochs.
- `CONFIDENCE_THRESHOLDS`: A list of confidence scores (e.g., 0.8, 0.9) used to determine if a sample can exit early. If the model's prediction confidence at an early exit surpasses a threshold, the sample exits.
- `DEVICE`: Automatically detects and uses a GPU (cuda) if available, otherwise defaults to the CPU.

Data Preparation

This section handles the loading and preprocessing of the CIFAR-10 dataset:

- `transform_train`: Defines a series of augmentations for the training data (random cropping, horizontal flips, rotation, color jitter) to improve generalization, followed by conversion to PyTorch tensors and normalization.
- `transform_test`: Defines transformations for the test data (conversion to PyTorch tensors and normalization).
- `trainset`, `testset`: Loads the CIFAR-10 training and testing datasets.
- `trainloader`, `testloader`: Creates data loaders to efficiently iterate through the datasets in mini-batches.

Model Definition: AdaptiveRes

This custom `AdaptiveRes` class extends `nn.Module` and modifies a pre-trained ResNet18 model:

- **Backbone**: Uses `torchvision.models.resnet18` as the base, adapted for CIFAR-10 (10 classes). The first convolutional layer (`conv1`) is modified for 32x32 input images, and `maxpool` is replaced with `nn.Identity()` as 32x32 images do not require initial pooling.

- **Exit 1:** An early exit point after backbone.layer2. It consists of adaptive average pooling, flattening, and a linear layer to produce 10 class logits.
- **Exit 2:** Another early exit point after backbone.layer3. Similar to Exit 1, it processes features and outputs 10 class logits.
- **forward method:** Defines the forward pass, computing outputs from Exit 1, Exit 2, and the final ResNet layer.

Training Loop

This section outlines the training procedure:

- **criterion:** Uses nn.CrossEntropyLoss for classification. The total loss is a sum of losses from all three exit points (o1, o2, o_final).
- **optimizer:** optim.Adam is used with a specified learning rate and weight decay.
- **scheduler:** torch.optim.lr_scheduler.CosineAnnealingLR adjusts the learning rate throughout training for better convergence.
- The loop iterates over epochs, performs forward and backward passes, updates model parameters, and steps the learning rate scheduler. The average loss and current learning rate are printed for each epoch.

Adaptive Inference & Metrics

This is the core evaluation section, demonstrating the adaptive inference strategy:

- **Evaluation Mode:** The model is set to `eval()` to disable dropout and batch normalization updates.
- **Batch Processing:** For each batch in the testloader:
 - Predictions and confidence scores are obtained from all three exits (`o1`, `o2`, `o_final`).
- For each sample in the batch, the model checks if the confidence at Exit 1 exceeds a given `CONFIDENCE_THRESHOLD`. If so, it classifies the sample and exits. If not, it proceeds to Exit 2. If Exit 2's confidence is sufficient, it exits. Otherwise, the sample proceeds to the final layer.
- **Metrics Calculation:**
 - `exit_counts`: Tracks how many samples exited at each stage (Exit 1, Exit 2, Final) for each confidence threshold.
 - `adaptive_res_correct`: Counts correct predictions for the adaptive inference strategy.
 - `exit_correct`: Counts correct predictions if **all** samples were forced to exit at a specific point (e.g., all at Exit 1, all at Exit 2, all at Final).

- `compute_ece`: A function to calculate Expected Calibration Error (ECE), which measures how well the predicted probabilities align with the actual accuracy.
- `benchmark_flops`: A function using `fvcore.nn.FlopCountAnalysis` to measure the computational cost (FLOPs) of executing up to each exit point. Custom `Exit1Model` and `Exit2Model` classes are used to precisely measure FLOPs for the intermediate exits.
- **Results Output:** The final table presents:
 - Top-1 Accuracy and its standard error.
 - Expected Calibration Error (ECE).
 - P-value (comparing adaptive performance to baseline final exit).
 - Computational Cost (MFLOPS).
 - Percentage of Compute Saved compared to the full model.
 - Distribution of samples exiting at each point.

How to Run

To run this notebook, execute the cells sequentially from top to bottom. The process involves:

1. **Imports:** Importing all necessary PyTorch, torchvision, and utility libraries.

2. **Configuration:** Setting hyperparameters for training and evaluation.
3. **Data Preparation:** Downloading and setting up the CIFAR-10 dataset and data loaders.
This step may take some time due to data download.
4. **Model Definition:** Defining the AdaptiveRes neural network architecture.
5. **Training:** The model will train for EPOCHS (default 30) on the CIFAR-10 training data.
This is the most time-consuming part.
6. **Evaluation:** After training, the model will be evaluated using the adaptive inference strategy on the test set, and various metrics will be calculated and printed.

Results Interpretation

The final output table provides a comprehensive comparison:

- **Adaptive Early Exit 1 / 2:** Shows accuracy, ECE, and FLOPs if all samples were forced to exit at Exit 1 or Exit 2, respectively. This gives a baseline for the performance of these intermediate parts of the network.
- **Baseline Final Exit:** Represents the performance of the full ResNet18 model without any early exits.
- **AdaptiveRes ($\theta = X.XX$):** These rows show the performance of the adaptive model for different confidence thresholds. Key metrics to observe are:
 - **Top-1 Accuracy:** How close it is to the Baseline Final Exit.

- **ECE:** A lower ECE indicates better calibration (i.e., confidence scores are reliable).
 - **Cost (MFLOPS) and Compute Saved:** Indicate the computational efficiency gained by using early exits.
 - **Exits (1, 2, Final):** Shows how many samples were processed by each exit, demonstrating the effectiveness of the early exit mechanism for different thresholds.
-

```
import torch
```

```
import torch.nn as nn
```

```
import torch.optim as optim
```

```
import torchvision
```

```
import torchvision.transforms as transforms
```

```
import torchvision.models as models
```

```
import time
```

```
import matplotlib.pyplot as plt
```

```
import numpy as np
```

```
from math import sqrt
```

```
# =====
```

```
# 1. CONFIGURATION
```

```
# =====
```

```
BATCH_SIZE = 256
```

```
LEARNING_RATE = 0.001
```

```
EPOCHS = 30
```

```
CONFIDENCE_THRESHOLDS = [0.8, 0.9, 0.95, 0.99]
```

```
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

```
print(f"Using device: {DEVICE}")
```

```
# =====
```

```
# 2. DATA PREPARATION
```

```
# =====
```

```
transform_train = transforms.Compose([
```

```
    transforms.RandomCrop(32, padding=4),
```

```
    transforms.RandomHorizontalFlip(),
```

```
transforms.RandomRotation(15),

transforms.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2, hue=0.1),

transforms.ToTensor(),

transforms.Normalize((0.4914, 0.4822, 0.4465), (0.2023, 0.1994, 0.2010)),

])
```

```
transform_test = transforms.Compose([

    transforms.ToTensor(),

    transforms.Normalize((0.4914, 0.4822, 0.4465), (0.2023, 0.1994, 0.2010)),

])
```

```
trainset = torchvision.datasets.CIFAR10(root='./data', train=True, download=True,

transform=transform_train)
```

```
trainloader = torch.utils.data.DataLoader(trainset, batch_size=BATCH_SIZE, shuffle=True,

num_workers=2)
```

```
testset = torchvision.datasets.CIFAR10(root='./data', train=False, download=True,

transform=transform_test)
```

```
testloader = torch.utils.data.DataLoader(testset, batch_size=BATCH_SIZE, shuffle=False,
num_workers=2)
```

```
# =====
```

```
# 3. MODEL DEFINITION
```

```
# =====
```

```
class AdaptiveRes(nn.Module):
```

```
    def __init__(self):
```

```
        super(AdaptiveRes, self).__init__()
```

```
        self.backbone = models.resnet18(num_classes=10)
```

```
        self.backbone.conv1 = nn.Conv2d(3, 64, kernel_size=3, stride=1, padding=1,
bias=False)
```

```
        self.backbone.maxpool = nn.Identity()
```

```
        # Exit 1 (After layer 2)
```

```
        self.exit1 = nn.Sequential(
```

```
            nn.AdaptiveAvgPool2d((1, 1)),
```

```
            nn.Flatten(),
```

```
nn.Linear(128, 10)

)

# Exit 2 (After layer 3)

self.exit2 = nn.Sequential(

    nn.AdaptiveAvgPool2d((1, 1)),

    nn.Flatten(),

    nn.Linear(256, 10)

)
```

```
def forward(self, x):
```

```
    x = self.backbone.conv1(x)
```

```
    x = self.backbone.bn1(x)
```

```
    x = self.backbone.relu(x)
```

```
    x = self.backbone.maxpool(x)
```

```
    x = self.backbone.layer1(x)
```

```
x = self.backbone.layer2(x)
```

```
out1 = self.exit1(x)
```

```
x = self.backbone.layer3(x)
```

```
out2 = self.exit2(x)
```

```
x = self.backbone.layer4(x)
```

```
x = self.backbone.avgpool(x)
```

```
x = torch.flatten(x, 1)
```

```
out_final = self.backbone.fc(x)
```

```
return out1, out2, out_final
```

```
model = AdaptiveRes().to(DEVICE)
```

```
print(model)
```

```
# =====
```

4. TRAINING LOOP (Upgraded Optimizer & Scheduler)

```
# =====
```

```
criterion = nn.CrossEntropyLoss()
```

```
optimizer = optim.Adam(model.parameters(), lr=LEARNING_RATE, weight_decay=1e-4)
```

```
scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=EPOCHS)
```

```
print(f"Starting Training ({EPOCHS} Epochs)...\nNOTE: Loss is displayed in standard Cross  
Entropy Loss scale to assess model quality.")
```

```
for epoch in range(EPOCHS):
```

```
    model.train()
```

```
    running_loss = 0.0
```

```
    for inputs, labels in trainloader:
```

```
        inputs, labels = inputs.to(DEVICE), labels.to(DEVICE)
```

```
        optimizer.zero_grad()
```

```
        o1, o2, o_final = model(inputs)
```

```
        loss = criterion(o1, labels) + criterion(o2, labels) + criterion(o_final, labels)
```

```
loss.backward()
```

```
optimizer.step()
```

```
running_loss += loss.item()
```

```
current_lr = scheduler.get_last_lr()[0]
```

```
scheduler.step()
```

```
print(f"Epoch {epoch+1}/{EPOCHS} | Loss: {running_loss / (3 * len(trainloader)):.4f} |  
LR: {current_lr:.6f}")
```

```
# =====
```

```
# 5. ADAPTIVE INFERENCE & METRICS
```

```
# =====
```

```
print("\nEvaluating...")
```

```
model.eval()
```

```
exit_counts = [[0, 0, 0], [0, 0, 0], [0, 0, 0], [0, 0, 0]]
```



```
adaptive_res_correct = [0, 0, 0, 0]
```

```
exit_correct = [0, 0, 0]
```

```
N_BINS = 10
```

```
calib_confs = [[], [], [], []]
```

```
calib_correct = [[], [], [], []]
```

```
exit_confs = [[], [], []]
```

```
exit_correct_flags = [[], [], []]
```

```
with torch.no_grad():
```

```
    for inputs, labels in testloader:
```

```
        inputs, labels = inputs.to(DEVICE), labels.to(DEVICE)
```

```
        o1, o2, o_final = model(inputs)
```

```
        p1 = torch.softmax(o1, dim=1)
```

```
        p2 = torch.softmax(o2, dim=1)
```

```
        max_p1, pred1 = torch.max(p1, 1)
```

```
max_p2, pred2 = torch.max(p2, 1)
```

```
_, pred_final = torch.max(o_final, 1)
```

```
# Adaptive logic for each sample in the batch
```

```
for i in range(len(labels)):
```

```
    sample_max_p1 = max_p1[i]
```

```
    sample_max_p2 = max_p2[i]
```

```
    sample_pred1 = pred1[i]
```

```
    sample_pred2 = pred2[i]
```

```
    sample_pred_final = pred_final[i]
```

```
    sample_label = labels[i]
```

```
    exit_confs[0].append(sample_max_p1.item())
```

```
    exit_correct_flags[0].append(int(sample_pred1 == sample_label))
```

```
    exit_confs[1].append(sample_max_p2.item())
```

```
    exit_correct_flags[1].append(int(sample_pred2 == sample_label))
```

```
final_conf = torch.softmax(o_final[i], dim=0).max().item()
```

```
exit_confs[2].append(final_conf)
```

```
exit_correct_flags[2].append(int(sample_pred_final == sample_label))
```

```
if sample_pred1 == sample_label:
```

```
    exit_correct[0] += 1
```

```
if sample_pred2 == sample_label:
```

```
    exit_correct[1] += 1
```

```
if sample_pred_final == sample_label:
```

```
    exit_correct[2] += 1
```

```
for threshold in range(0, len(CONFIDENCE_THRESHOLDS)):
```

```
    if sample_max_p1.item() > CONFIDENCE_THRESHOLDS[threshold]:
```

```
        final_pred_sample = sample_pred1
```

```
        final_conf_sample = sample_max_p1.item()
```

```
        exit_counts[threshold][0] += 1
```

```
elif sample_max_p2.item() > CONFIDENCE_THRESHOLDS[threshold]:
```

```
    final_pred_sample = sample_pred2
```

```
    final_conf_sample = sample_max_p2.item()
```

```
    exit_counts[threshold][1] += 1
```

```
else:
```

```
    final_pred_sample = sample_pred_final
```

```
    final_conf_sample = final_conf
```

```
    exit_counts[threshold][2] += 1
```

```
if final_pred_sample == sample_label:
```

```
    adaptive_res_correct[threshold] += 1
```

```
calib_confs[threshold].append(final_conf_sample)
```

```
calib_correct[threshold].append(int(final_pred_sample == sample_label))
```

```
def compute_ece(confs, corrects, n_bins=10):
```

```
    confs = np.array(confs)
```

```
corrects = np.array(corrects)
```

```
bins = np.linspace(0, 1, n_bins + 1)
```

```
ece = 0.0
```

```
for i in range(n_bins):
```

```
    mask = (confs > bins[i]) & (confs <= bins[i+1])
```

```
    if np.any(mask):
```

```
        acc = corrects[mask].mean()
```

```
        avg_conf = confs[mask].mean()
```

```
        ece += np.abs(acc - avg_conf) * mask.mean()
```

```
return ece * 100
```

```
adaptive_res_acc = np.array(adaptive_res_correct) / 100
```

```
exit_acc = np.array(exit_correct) / 100
```

```
# !pip install fvcore
```

```
from fvcore.nn import FlopCountAnalysis
```

```
import torch
```

```
def benchmark_flops(model):
```

```
    model.eval()
```

```
    inputs = torch.randn(1, 3, 32, 32).to(DEVICE)
```

```
    # Sub-models to measure exactly where the execution stops
```

```
    # 1. FLOPs for Exit 1
```

```
    class Exit1Model(torch.nn.Module):
```

```
        def __init__(self, full_model):
```

```
            super().__init__()
```

```
            self.m = full_model
```

```
        def forward(self, x):
```

```
            x = self.m.backbone.conv1(x)
```

```
            x = self.m.backbone.bn1(x)
```

```
            x = self.m.backbone.relu(x)
```

```
x = self.m.backbone.maxpool(x)
```

```
x = self.m.backbone.layer1(x)
```

```
x = self.m.backbone.layer2(x)
```

```
return self.m.exit1(x)
```

2. FLOPs for Exit 2

```
class Exit2Model(torch.nn.Module):
```

```
    def __init__(self, full_model):
```

```
        super().__init__()
```

```
        self.m = full_model
```

```
    def forward(self, x):
```

```
        x = self.m.backbone.conv1(x)
```

```
        x = self.m.backbone.bn1(x)
```

```
        x = self.m.backbone.relu(x)
```

```
        x = self.m.backbone.maxpool(x)
```

```
        x = self.m.backbone.layer1(x)
```

```
        x = self.m.backbone.layer2(x)
```

```
x = self.m.backbone.layer3(x)
```

```
return self.m.exit2(x)
```

```
# Calculate
```

```
flops1 = FlopCountAnalysis(Exit1Model(model), inputs).total()
```

```
flops2 = FlopCountAnalysis(Exit2Model(model), inputs).total()
```

```
flops_final = FlopCountAnalysis(model, inputs).total()
```

```
return [flops1, flops2, flops_final]
```

```
flop_values = benchmark_flops(model)
```

```
# pip install scipy
```

```
import scipy.stats as stats
```

```
adaptive_res_cost = []
```

```
for exit_count in exit_counts:
```



```

adaptive_res_cost.append(np.sum(np.array(flop_values) * np.array(exit_count) / 10000))

exit1_standard_error = sqrt(abs(((exit_acc[2] / 100.00) * (1.00 - (exit_acc[2] / 100.00))) /
10000.00))

exit2_standard_error = sqrt(abs(((exit_acc[2] / 100.00) * (1.00 - (exit_acc[2] / 100.00))) /
10000.00))

final_standard_error = sqrt(abs(((exit_acc[2] / 100.00) * (1.00 - (exit_acc[2] / 100.00))) /
10000.00))

adaptive_res_standard_error = []

for threshold in range(0, len(CONFIDENCE_THRESHOLDS)):

    adaptive_res_standard_error.append(sqrt(abs(((adaptive_res_acc[threshold] / 100.00) *
(1.00 - (adaptive_res_acc[threshold] / 100.00))) / 10000.00)))

adaptive_res_z = ((adaptive_res_acc / 100.00) - (exit_acc[2] / 100.00)) /
final_standard_error

adaptive_res_p = stats.norm.sf(abs(adaptive_res_z))

print(f"Final Results:")

print(f"Total Samples: 10000")

```

```
print("+-----+  
-----+")
```

```
print("| Model Execution Path | Top-1 Accuracy | ECE | P value | Cost (MFLOPS) |  
Compute Saved | Exits (1, 2, Final) |")
```

```
print("+-----+  
-----+")
```

```
print(f"| Adaptive Early Exit 1 | {exit_acc[0]:.2f}% +/- {(exit1_standard_error *  
100.00):.2f}% | {compute_ece(exit_confs[0], exit_correct_flags[0], N_BINS):.2f}% | N/A |  
{flop_values[0]/1e6:.2f} MFLOPS | {(100.00 - (100 * flop_values[0] / flop_values[2])):.2f}%  
| [10000, 0, 0] |")
```

```
print(f"| Adaptive Early Exit 2 | {exit_acc[1]:.2f}% +/- {(exit2_standard_error *  
100.00):.2f}% | {compute_ece(exit_confs[1], exit_correct_flags[1], N_BINS):.2f}% | N/A |  
{flop_values[1]/1e6:.2f} MFLOPS | {(100.00 - (100 * flop_values[1] / flop_values[2])):.2f}%  
| [0, 10000, 0] |")
```

```
print(f"| Baseline Final Exit | {exit_acc[2]:.2f}% +/- {(final_standard_error *  
100.00):.2f}% | {compute_ece(exit_confs[2], exit_correct_flags[2], N_BINS):.2f}% | N/A |  
{flop_values[2]/1e6:.2f} MFLOPS | {(100.00 - (100 * flop_values[2] / flop_values[2])):.2f}%  
| [0, 0, 10000] |")
```

```
print(f"| AdaptiveRes ( $\theta$  = {CONFIDENCE_THRESHOLDS[0]:.2f}) |  
{adaptive_res_acc[0]:.2f}% +/- {(adaptive_res_standard_error[0] * 100.00):.2f}% |  
{compute_ece(calib_confs[0], calib_correct[0], N_BINS):.2f}% | {adaptive_res_p[0]:.2f} |
```

```

{adaptive_res_cost[0]/1e6:.2f} MFLOPS | {(100 - (100 * adaptive_res_cost[0] /
flop_values[2])):.2f}%    | {exit_counts[0]} |")

print(f"| AdaptiveRes ( $\theta$  = {CONFIDENCE_THRESHOLDS[1]:.2f}) |
{adaptive_res_acc[1]:.2f}% +/- {(adaptive_res_standard_error[1] * 100.00):.2f}% |
{compute_ece(calib_confs[1], calib_correct[1], N_BINS):.2f}% | {adaptive_res_p[1]:.2f} |
{adaptive_res_cost[1]/1e6:.2f} MFLOPS | {(100 - (100 * adaptive_res_cost[1] /
flop_values[2])):.2f}%    | {exit_counts[1]} |")

print(f"| AdaptiveRes ( $\theta$  = {CONFIDENCE_THRESHOLDS[2]:.2f}) |
{adaptive_res_acc[2]:.2f}% +/- {(adaptive_res_standard_error[2] * 100.00):.2f}% |
{compute_ece(calib_confs[2], calib_correct[2], N_BINS):.2f}% | {adaptive_res_p[2]:.2f} |
{adaptive_res_cost[2]/1e6:.2f} MFLOPS | {(100 - (100 * adaptive_res_cost[2] /
flop_values[2])):.2f}%    | {exit_counts[2]} |")

print(f"| AdaptiveRes ( $\theta$  = {CONFIDENCE_THRESHOLDS[3]:.2f}) |
{adaptive_res_acc[3]:.2f}% +/- {(adaptive_res_standard_error[3] * 100.00):.2f}% |
{compute_ece(calib_confs[3], calib_correct[3], N_BINS):.2f}% | {adaptive_res_p[3]:.2f} |
{adaptive_res_cost[3]/1e6:.2f} MFLOPS | {(100 - (100 * adaptive_res_cost[3] /
flop_values[2])):.2f}%    | {exit_counts[3]} |")

print("+-----+
-----+")

```